

Manual de montaje y operacion del pipeline

Mortalidad y AVP por causa especifica

Table of contents

1	Objetivo	2
2	Arquitectura operativa	2
2.1	Que queda dentro del repositorio	2
2.2	Que puede vivir fuera del repositorio	2
2.3	Principios	2
3	Punto unico de configuracion	3
3.1	Archivo principal	3
3.2	Variables de entorno soportadas	3
3.2.1	Raw principal	3
3.2.2	Datasets externos	3
3.3	Regla practica	4
4	Requisitos de sistema	4
5	Procedimiento paso a paso	4
5.1	1. Clonar y ubicar el proyecto	4
5.2	2. Montar el raw y externos	4
5.3	3. Ejecutar preflight	5
5.4	4. Correr el pipeline	5
5.4.1	Corrida completa	5
5.4.2	Solo core	5
5.4.3	Solo metodos	5
5.4.4	Solo reportes	5
5.5	5. Validar baseline	6
6	Runner y scripts canonicos	6
7	Semantica de carpetas del proyecto	6
7.1	Codigo	6
7.2	Salidas	6
8	Politica de nombres	7
8.1	Capa oficial	7

8.2	Rama historica	7
9	Interpretacion de salidas de control	7
9.1	Preflight	7
9.2	Corrida del pipeline	7
10	Politica operativa de warnings	8
10.1	Warnings bloqueantes	8
10.2	Warnings tolerados pero documentados	8
10.3	Warnings que deben revisarse si reaparecen	8

1 Objetivo

Este manual describe como montar y ejecutar el proyecto en:

- laptop local
- servidor nuevo
- contenedor Docker

La meta es que un analista pueda cambiar un solo punto de configuracion para localizar el raw y los datasets externos, correr el preflight, ejecutar el pipeline y validar resultados sin editar scripts metodologicos.

2 Arquitectura operativa

2.1 Que queda dentro del repositorio

- codigo del pipeline
- configuracion del proyecto
- outputs derivados y finales
- reportes
- baseline de validacion

2.2 Que puede vivir fuera del repositorio

- raw SINADef
- datasets externos declarados en `config/external_sources.yml`
- raw montado en disco protegido o ruta de solo lectura

2.3 Principios

- los inputs externos se tratan como inmutables
- los outputs se escriben dentro del proyecto
- la resolucion de paths se hace desde un helper central
- no se deben editar scripts metodologicos para cambiar rutas

3 Punto unico de configuracion

La resolucion de inputs sigue esta precedencia:

1. variables de entorno
2. config/runtime_paths.yml
3. fallback al layout local del repo

3.1 Archivo principal

- config/runtime_paths.yml

Contenido esperado:

```
raw_root: ""
external_root: ""

inputs:
  sinadev_dir: ""

external_dataset_overrides:
  population_result: ""
  life_table_mortality_single_age: ""
  life_table_standard_single_age: ""
```

3.2 Variables de entorno soportadas

3.2.1 Raw principal

- MCE_SINADEF_DIR
- MCE_RAW_ROOT

3.2.2 Datasets externos

- MCE_EXTERNAL_ROOT
- MCE_EXT_POPULATION_RESULT_PATH
- MCE_EXT_LIFE_TABLE_MORTALITY_SINGLE_AGE_PATH
- MCE_EXT_LIFE_TABLE_STANDARD_SINGLE_AGE_PATH

3.3 Regla practica

Si el raw esta en otro disco o montaje protegido:

- montar el raw
- definir MCE_SINADF_DIR o MCE_RAW_ROOT
- definir MCE_EXTERNAL_ROOT si los externos tambien viven fuera del repo
- correr preflight

4 Requisitos de sistema

- R disponible en linea de comandos
- Quarto instalado
- permisos de lectura sobre raw y externos
- permisos de escritura sobre:
 - data/final
 - data/derived
 - data/_catalog
 - reports
 - outputs

Directorios recomendados de solo lectura:

- raw externo
- datasets externos

5 Procedimiento paso a paso

5.1 1. Clonar y ubicar el proyecto

Ubicar el repo en un directorio escribible del servidor o contenedor.

5.2 2. Montar el raw y externos

Ejemplo Linux/Docker:

```
export MCE_SINADF_DIR=/mnt/protected/raw/sinadef
export MCE_EXTERNAL_ROOT=/mnt/projects
```

Ejemplo PowerShell:

```
$env:MCE_SINADF_DIR="D:\\raw_protegido\\sinadef"
$env:MCE_EXTERNAL_ROOT="D:\\repos_datos"
```

5.3 3. Ejecutar preflight

```
Rscript ./scripts/run_preflight_checks.R
```

Revisar:

- data/derived/qc/run_pipeline/preflight_checks.csv
- data/derived/qc/run_pipeline/preflight_summary.csv

Condicion de aprobacion:

- sin fallos bloqueantes

5.4 4. Correr el pipeline

5.4.1 Corrida completa

```
Rscript ./scripts/run_pipeline.R --profile full --clean-first
```

5.4.2 Solo core

```
Rscript ./scripts/run_pipeline.R --profile core
```

5.4.3 Solo metodos

```
Rscript ./scripts/run_pipeline.R --profile methods
```

5.4.4 Solo reportes

```
Rscript ./scripts/run_pipeline.R --profile reports
```

5.5 5. Validar baseline

```
Rscript ./scripts/compare_validation_baseline.R
```

Revisar:

- data/derived/qc/baseline_compare/qc_baseline_vs_rerun_summary.csv

Condicion de aprobacion:

- cero diferencias sustantivas

6 Runner y scripts canonicos

La operacion recomendada usa:

- scripts/run_preflight_checks.R
- scripts/run_pipeline.R

La capa semantica oficial del runner esta documentada en:

- config/pipeline_steps.csv

Esto permite operar y documentar con nombres canonicos en la estructura final limpia del repo.

7 Semantica de carpetas del proyecto

7.1Codigo

- scripts/: entrypoints operativos de uso regular.
- R/: helpers y modulos compartidos.
- R/diagnostics/: diagnosticos reproducibles fuera del rerun normal.
- R/maintenance/: auditoria, gobernanza y mantenimiento no operativo diario.
- reports/: constructores de salidas editoriales y reportables.
- config/: configuracion y soporte.

7.2 Salidas

- data/derived/qc/: QC tabular canonico del pipeline.
- reports/: productos editoriales y reportables.
- outputs/: auxiliares no tabulares y no editoriales.

Las rutas QC y auxiliares activas usan solo nombres semanticos canonicos.

8 Politica de nombres

8.1 Capa oficial

- scripts canonicos sin prefijo numerico
- `run_preflight_checks.R`
- `run_pipeline.R`
- `script_path_canonical` en `config/pipeline_steps.csv`

8.2 Rama historica

- la referencia historica completa vive en la rama `deprecated/full-history`
- `main` es la unica rama recomendada para operacion y mantenimiento corriente

9 Interpretacion de salidas de control

9.1 Preflight

Archivo principal:

- `data/derived/qc/run_pipeline/preflight_checks.csv`

Que revisar:

- raw principal resuelto
- externos obligatorios legibles
- directorios de salida escribibles
- baseline disponible

9.2 Corrida del pipeline

Archivo principal:

- `data/derived/qc/run_pipeline/pipeline_run_log.csv`

Que revisar:

- todos los steps con `status = success`
- tiempos de ejecucion
- script canonico ejecutado

10 Politica operativa de warnings

10.1 Warnings bloqueantes

- fallos de preflight
- steps del runner con `status != success`
- diferencias sustantivas en la comparacion baseline

10.2 Warnings tolerados pero documentados

- outliers de `y11_rate` asociados a denominadores muy pequenos en edades extremas
- fallback `parquet -> csv` del mismo dataset canonico cuando el reporte puede continuar sin cambiar la ubicacion de lectura

10.3 Warnings que deben revisarse si reaparecen

- reciclado de `data.table` en scripts metodologicos
- warnings de maquetacion del Word final

Registro vigente:

- `docs/minor_observations_status.csv`